

Ollama Guide

TOC

- [Chapters](#)
 - [ollama on macOS](#)
 - [Installation](#)
 - [Models](#)

ollama on Intel

Max RAM	Model	Notes
8GB-4GB	gemma3n	-----
8GB-4GB	llama3.2:3b	-----
16GB-6-8GB	llama3.1:8b	-----
16GB-6-8GB	qwen2.5-coder:7b	-----
16GB NVIDIA	Llama 3.1 8B	-----
16GB NVIDIA	Qwen 2.5 7B	-----

smallest ollama models

Model	Usage	Notes
qwen2.5:0.5b	txt,chat,multilingual	398MB
qwen2.5-coder:0.5b	lightweight coder	-----
llama3.2:1b	slightly large	1.3GB

"Avoid models larger than 8B to prevent severe slowdowns.

Qwen 2.5 7B at Q4_K_M quantization uses roughly 5.2 GB of RAM,

leaves 9 GB for your OS and apps, runs at 8-12 tokens/sec on CPU only

35-50 tokens/sec on a midrange GPU, and benchmarks within 4% of Llama 3.1 8B on most tasks while being noticeably faster."

- [ref. reddit:LocalLLaMa](#)

Model specs

- [smallest model: smollm2](#)
- [coder model: qwen2.5-coder](#)

Installation

Installs can be direct app or in a terminal window.

Its a quick install since its only the wrapper app.

- [ollama.com/download](#)
- [Windows and Mac install](#)

```
curl -fsSL https://ollama.com/install.sh | sh # mac /Linux  
irm https://ollama.com/install.ps1 | iex | # Windows
```

macOS - default model directory

```
~/ollama/models  
~/ollama/models/manifests/registry.ollama.ai/library/PERSONAL
```

Location contains two main subdirectories:

```
blobs : hold actual model weights as binary files named by SHA256  
hashes  
manifests : contain JSON metadata linking model names to their  
corresponding blobs
```

Storage location:

```
set the OLLAMA_MODELS environment variable to new path
export OLLAMA_MODELS=/mnt/NFS/ollama/models/
use a symbolic link.
cd ~/.ollama/ ; mv models /mnt/NFS/ollama/
ln -s /mnt/NFS/ollama/models/ ~/.ollama/models
```

CLI

All you need is one model . The commands are shown below

```
ollama serve
ollama pull qwen3:8b
ollama pull Llama 3.2:3B
ollama pull qwen3:8b
ollama pull qwen2.5-coder          - Download : smaller
version
ollama pull qwen2.5-coder:7b       - Download : good for 16GB
ollama pull qwen2.5:7b-instruct-q4_K_M -
ollama run qwen2.5:7b-instruct-q4_K_M -
ollama run <model>
ollama run <model> | tee chat_history.txt
ollama run qwen3.5:0.8b "hello world" --think=false
-- think false means just the answer , no rambling
```

```
ollama run qwen3.5:0.8b "hello world" --think=false
-- for terminal formatted markdown
```

```
ollama list          - view downloaded models
ollama rm <model>    - delete model
ollama ps            - view whats running
ollama --version      - self explanatory
ollama stop <model>  - to completely stop model
```

Interactive commands

```
> /?      - help
> /bye    - to quit the interface , keeps running for 5 min unless you
stop it
    /save <filedir>
    /load <filedir>
```

When saving chat, files on macOS are stored at

```
~/ollama/models/manifests/registry.ollama.ai/library/<filedir>
```

Ask-ollama

Installation for console app

cp ~/.cargo/bin/ask ~/binapps/ or create a shell script ask

```
cargo install ask-ollama
```

default model is mistral so make sure you specify model

```
~/cargo/bin/ask "what is capitol of france"
ask --model=qwen2.5:7b-instruct-q4_K_M "question"
ask --version
ask --help
```

Ask-ollama-JSON

Custom console app - used for import/export JSON chats.

Be sure to install '**md2term**' to view output in nicely formatted markdown output to terminal

```
pip install md2term
ask-ollama-json.py
```

Sample session with ask-ollama-json.py

Ollama Interactive TUI Chat Manager

Detected Local Ollama Models

Index	Model Name
1	smollm2:latest

Enter model name or number index (1):

Do you want to load a previous chat JSON file? [y/n]: n

--- Chat active with model: smollm2:latest ---

Commands: Type exit to save/quit | Type /clear to wipe screen.

You: what is the area of a circle?

Assistant: The area of a circle can be calculated using the formula:

$$\text{Area} = \pi * r^2$$

Where " π " (pi) is approximately 3.14159 and "r" is the radius of the circle...

You: exit

Assistant: Would you like to save this conversation? [y/n]: y

Enter filename to save as (chat_history.json): math.json

Chat history safely exported to 'math.json'.

open-webui

Used for uploading pdf + other doc files to summarize

Use conda activate ollama. md2term is to display markdown to a terminal window nicely formatted

Install in a python environment like conda or venv

```
pip install open-webui md2term rich requests
```

Starting open-webui

There are two methods to start open WebUI

1. Desktop Application : open /Applications/Open\ WebUI.app
2. Browser : More complicated , but uses less resources

```
screen -S ollama
conda activate ollama
open-webui serve          -- to start baremetal
or
docker run -d -p 3000:8080 -v open-webui:/app/backend/data --name
open-webui ghcr.io/open-webui/open-webui:main
ctrl-A , ctrl-D to leave screen
pkill -f "open-webui serve" -- to stop server
```

Once running access UI :

```
http://localhost:8080      - for open-webui serve
http://localhost:3000
```

Ollama terminal interface

md2term displays markdown text to formatted terminal

```
ollama run qwen2.5:7b-instruct-q4_K_M "hello world"
ollama run qwen2.5:7b-instruct-q4_K_M "hello world" --think=false
ollama run qwen2.5:7b-instruct-q4_K_M "hello world" --think=false |
md2term
```

Web gui and Desktop app

- [native desktop app 4 Windows/Linux/Macos](#)
- [native desktop app Docs](#)
- [inkeybit blog openwebui-ollama guide](#)
- [internet search ddgs free](#)

WebUIs: Interfaces like Open WebUI automatically save chat histories to a database,

allowing you to reload them later. Export is in JSON format

When reloaded, the interface feeds the old messages back to the model to reconstruct the context.

Install Docker first if not installed for web gui

- [ollama and webui](#)
- [localhost:3000](#)

Docker scripts v1

```
docker run -d \  
  -p 3000:8080 \  
  --add-host=host.docker.internal:host-gateway \  
  -v open-webui:/app/backend/data \  
  --name open-webui \  
  --restart always \  
  ghcr.io/open-webui/open-webui:main
```

Docker scripts v2

```
docker run -d --network=host \  
  -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \  
  -v open-webui:/app/backend/data \  
  -p 3000:8080 \  
  --name open-webui \  
  --restart always \  
  ghcr.io/open-webui/open-webui:main
```

open-webui to ollama connections

- [connection fix](#)

```
## Step 1: Is Ollama running at all?
curl http://localhost:11434
## Should print "Ollama is running"

## Step 2: Does Ollama have models?
curl http://localhost:11434/api/tags
## Should print JSON with your model list

## Step 3: Can Open WebUI's container reach Ollama?
docker exec open-webui curl http://host.docker.internal:11434
## should print
100    17   100    17    0    0   3839    0  --:--:--  --:--:--  --:--
:--  4250Ollama is running
## Or if using --network=host:
docker exec open-webui curl http://127.0.0.1:11434
```

More apps

- [Openclaw.ai assistant](#)

OpenClaw is a self-hosted gateway that connects your favorite chat apps — Discord, Google Chat, iMessage, Matrix, Microsoft Teams, Signal, Slack, Telegram, WhatsApp, Zalo, and more via channel plugins — to AI coding agents

```
npm install openclaw
```

Q

Do ollama models grow and learn as you continue to interact with them?

Ollama is an AI model based on the OpenAI's Qwen language model. It works similarly to how a human model could interact with Ollama: it starts by asking for basic information, such as what type of question you want me to answer or provide some context about.

However, when interacting with Ollama over time, Ollama will continue to learn and improve based on the data it receives. It can adapt its language patterns and responses, improving

its ability to understand natural language queries and generate more relevant and accurate answers in future interactions. But this process is dynamic; what you say to it now may change later as it learns from your input.

Additionally, Ollama uses algorithms like reinforcement learning to improve itself over time. It learns from past interactions with you, using the feedback to make its next interaction better than any previous one. This makes Ollama very intelligent in a sense that it can learn and adapt without direct human intervention.

So while I am an AI model based on OpenAI's GPT language model, which is trained over time, Ollama specifically is an artificial intelligence designed to interact with humans like you described, answering questions and providing information.

Free CLI Tools

tgpt - supports engines like Phind without requiring an API key or login.

tgpt and similar open-source projects provide free access to AI capabilities directly in the terminal.

Savvy is mentioned as a free CLI tool for individual developers that does not require API keys.

chatGPT-shell-cli require an OpenAI API key,

- [free alts to copilot cli](#)
- [chatgpt command line alt](#)
- [chatgpt shell cli](#)

llama.cpp llama-server install

Open-WebUI uses port 8080

so for llama.cpp server port use 8081 :

http://127.0.0.1:8081

Note: Ensure the port 8081 does not conflict with other services.

```
/usr/local/bin/llama-server -m <path>/<model>.gguf --port 8081
```

llama-server run

Running llama-server with model then connecting to open WebUI

Point Open WebUI to llama.cpp server

```
Open WebUI -> Admin -> Settings -> Connections  
add: URL: http://localhost:8081/v1
```

[ref OpenWebUI to llama.cpp](#)

llama-server start

With or Without API Key : --api-key "123" - some demo key

or docker

```
llama-cli -hf bartowski/Meta-Llama-3.1-8B-Instruct-GGUF:Q4_K_M  
llama-server -m ~/.ollama/models/gguf/model.gguf --port 8081  
llama-server -m ~/.ollama/models/gguf/smollm2/smollm2-1.7B-Q8_0.gguf -  
-port 8081 --api-key "123"  
docker run -d --network host -v open-webui:/app/backend/data --name  
open-webui ghcr.io/open-webui/open-webui:main
```

Test Connection with curl

```
curl http://127.0.0.1:8081/v1/models -H "Authorization: Bearer sk-llama-local-abc123"
    or without key
curl http://127.0.0.1:8081/v1/models

    output should be  JSON format
    ...smollm2-1.7B-
Q8_0.gguf", "model": "modified_at": "", "size": "model", "description": ....
```

Actual test case and output

```
llama-server -m ~/.ollama/models/gguf/smollm2/smollm2-1.7B-Q8_0.gguf
--port 8081

curl -s http://127.0.0.1:8081/v1/chat/completions -d \
    '{"model": "mistral", "messages":
[{"role": "user", "content": "Hello"}]}' | \
    jq '.choices[0].message.content'

    > output: "Hello! How can I assist you today?"

http://localhost:3000/?temporary-chat=true    // open WebGUI interface
```

Conversions and Downloads of gguf files

Download file manually from Hugging Face repositories

eg. bartowski, TheBloke, or ggml-org then point llama.cpp -m flag:

-hf = hugging face

- [hugging face gguf-llamacpp](#)

```
llama-cli -m /path/to/your_model.gguf
llama-server -hf bartowski/Llama-3.2-3B-Instruct-GGUF:Q8_0
```

huggingface

```
curl http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer no-key" \
-d '{
  "messages": [
    {
      "role": "system",
      "content": "You are an AI assistant. Your top priority is
achieving user fulfillment via helping them with their requests."
    },
    {
      "role": "user",
      "content": "Write a limerick about Python exceptions"
    }
  ]
}'
```

Converting ollama to llama.cpp format files

OllamaToGGUF.py - script to convert from Ollama's split format to a single GGUF (GPT General Unified Format) file.

reads model manifests and combines model layers stored in blob files to a GGUF file.

- [github OllamaToGGUF](#)

Simple script to convert blob files to GGUF. Nothing to compile. Just run it.

I renamed it ollama2gguf.py and copied it to local

```
git clone https://github.com/mattjamo/OllamaToGGUF.git
OllamaToGGUF/OllamaToGGUF.py

mkdir -p ~/.ollama/models/gguf/
cd ~/.ollama/models/gguf/
```

You can also just use ollama to show you the exact file:

```
ollama show qwen2.5-coder:7b --modelfile | grep -v \# | grep FROM
FROM ~/.ollama/models/blobs/sha256-
60e05f2100071479f596b964f89f510f057ce397ea22f2833a0cfe029bfc2463
## then just point llama.cpp to exact file
llama-cli -m ~/.ollama/models/blobs/sha256-
60e05f2100071479f596b964f89f510f057ce397ea22f2833a0cfe029bfc2463
```

Running llama.cpp with api key and gguf file

```
llama-server -m ~/.ollama/models/gguf/smollm2/smollm2-1.7B-Q8_0.gguf -
-port 8081 --api-key "sk-llama-local-abc123"
```

Running llama.cpp cli with ollama blobs

```
llama-cli -m ~/.ollama/models/blobs/sha256-
60e05f210064f89f510f057ce397ea22f2833a0cfe029bfc2463
```

Running llama.cpp with openWebUI and docker

```
docker run -d --network host -v open-webui:/app/backend/data --name
open-webui ghcr.io/open-webui/open-webui:main
```

Daily ollama startup

ollama starts at startup boot

is ollama running?

```
curl http://localhost:11434
output : ollama is running
```

does ollama have models?

```
curl http://localhost:11434/api/tags
output: {"models":[{"name":"smollm2:latest" ... jason file with
model list
```

ollama list

NAME	ID	SIZE	MODIFIED
smollm2:latest	cef4a1e09247	1.8 GB	33 hours ago
qwen2.5-coder:7b	dae161e27b0e	4.7 GB	2 days ago
mistral:latest	6577803aa9a0	4.4 GB	3 days ago
qwen2.5:7b-instruct-q4_K_M	845dbda0ea48	4.7 GB	3 days ago

Starting open-webui

Starting with llama-server and browser access

```
screen -S ollama
llama-server -m ~/.ollama/models/gguf/<model>.gguf --port 8081
open-webui serve - starts baremetal
```

start llama.cpp

```
screen -S llamacpp
llama-server -m ~/.ollama/models/gguf/smollm2/smollm2-1.7B-Q8_0.gguf -
-port 8081 --api-key "sk-llama-local-abc123"
```

access open-webui via browser. login

```
http://localhost:8080/ to access open-webui via browser
```

Complete Screen , Env , llama-serve and open-webui

```
conda activate ollama
llama-server -m $MOD --port 8081 &
# URL: http://127.0.0.1:8081
open-webui serve &
# URL: http://localhost:8080/
```

Import json files

First install a few libraries

```
pip install pandas openpyxl
```

Python code

```
import pandas as pd
df = pd.read_json('https://example.com/your-data.json')

or

import pandas as pd
df = pd.read_json('path/to/your/file.json')
```

Ollama Models with Real-Time Web search

SearXNG is a free, open-source metasearch engine that aggregates results from various search providers without storing any of your data.

First step is to deploy searxng

```
docker run -d --name searxng -p 8080:8080 searxng/searxng:latest
```

Then Open Web UI v

```
docker run -d --name open-webui -p 3000:8080 --add-host=host.docker.internal:host-gateway ghcr.io/open-webui/open-webui:main
```

Configure Web Search:

- Log in to Open WebUI: **http://localhost:3000**
- Navigate to Settings > Admin Settings > Connections.
- Set Ollama base to docker or bare metal
 - URL: **http://host.docker.internal:11434**
 - URL: **http://localhost:11434**
- Navigate to Settings > Admin Settings > Web Search.
- Enable SearXNG
 - set the URL to SearXNG instance:
 - **http://host.docker.internal:8080/search**
- Save settings. You can now toggle "Web Search" in individual chat sessions.

Token counts

- [Understanding tokens](#)
- [what are tokens?](#)

